Python OOP: Encapsulation

Focused practice problems covering Private & Protected Attributes, Getters/Setters, Property Decorators, and Data Hiding without code solutions.

DATA HIDING | PRIVATE ATTRIBUTES | GETTERS & SETTERS | @PROPERTY DECORATOR

Task 1: Bank Account Vault

Private Attributes

PROBLEM STATEMENT

Design a secure bank account class using strict private attributes:

1. Create a class `BankAccount` initializing with public `account_holder` and private `__balance` (float).
2. Implement `get_balance()` to return the private balance.
3. Implement `deposit(amount)` to increase balance if `amount > 0`, otherwise print `"Invalid deposit amount"`.
4. Implement `withdraw(amount)` to deduct funds if `0 < amount <= __balance`, else print `"Insufficient funds or invalid amount"`.

SAMPLE INPUT

```
Account: "Alice", Initial Balance: 500.0 Actions: - deposit(200.0) - withdraw(800.0) - withdraw(150.0) - get_balance()
```

SAMPLE OUTPUT

```
Insufficient funds or invalid amount 550.0
```

Task 2: Employee Salary Guard

Getters & Setters

PROBLEM STATEMENT

Design an employee payroll class that controls salary updates:

1. Create a class `Employee` initializing with `name` and private attribute `__salary`.
2. Implement explicit getter `get_salary()` returning the current salary.
3. Implement explicit setter `set_salary(new_salary)`. Salary must be a positive number greater than or equal to 30000.0. If invalid, print `"Invalid salary update"` without changing the existing value.

SAMPLE INPUT

```
Employee: "Bob", Salary: 45000.0 Actions: - set_salary(20000.0) - set_salary(55000.0) - get_salary()
```

SAMPLE OUTPUT

```
Invalid salary update 55000.0
```

Task 3: Student Grade Property

Property Decorator

PROBLEM STATEMENT

Manage student test marks using Python's `@property` decorator:

1. Create a class `StudentMark` initializing with `student_name` and private `__score` (0 to 100).
2. Define a getter property `@property def score(self)` returning `__score`.
3. Define a setter property `@score.setter` that updates `__score` only if the value is between 0 and 100 inclusive. If out of range, print `"Score must be between 0 and 100"`.

SAMPLE INPUT

```
Student: "Charlie", Initial Score: 85 Actions: - student.score = 105 - student.score = 92 - print(student.score)
```

SAMPLE OUTPUT

```
Score must be between 0 and 100 92
```

Task 4: Temperature Converter with Property

Property Decorator

PROBLEM STATEMENT

Design a temperature tracking class enforcing absolute zero bounds:

1. Create a class `Temperature` initializing with private `__celsius`.
2. Define property `celsius` with getter and setter. The setter must reject temperatures below absolute zero (`-273.15°C`), printing `"Temperature below absolute zero is invalid"`.
3. Define a read-only property `fahrenheit` (getter only) that dynamically computes and returns `(celsius * 9/5) + 32`.

SAMPLE INPUT

```
Initial Celsius: 25.0 Actions: - temp.celsius = -300.0 - print(temp.fahrenheit) - temp.celsius = 0.0 - print(temp.fahrenheit)
```

SAMPLE OUTPUT

```
Temperature below absolute zero is invalid 77.0 32.0
```

Task 5: User Authentication Credentials

Data Hiding

PROBLEM STATEMENT

Design a user login profile protecting sensitive password storage:

1. Create a class `UserProfile` initializing with public `username` and private `__password`.
2. Implement method `check_password(input_password)` returning `True` if input matches `__password`, else `False`.
3. Implement method `change_password(old_password, new_password)`. If `old_password` is verified and `len(new_password) >= 8`, update password and return `"Password updated"`, otherwise return `"Password change failed"`.

SAMPLE INPUT

```
User: "david_dev", Pass: "secret123" Actions: - change_password("wrongpass", "newpass123") - change_password("secret123", "short") - change_password("secret123", "securePass2026")
```

SAMPLE OUTPUT

```
Password change failed Password change failed Password updated
```

Task 6: Vehicle Speed Governor

Protected & Private

PROBLEM STATEMENT

Design a vehicle monitoring system restricting maximum speed:

1. Create a class `Car` initializing with `model`, protected `_max_speed` (default 180), and private `__current_speed` (default 0).
2. Implement `accelerate(increment)` to increase `__current_speed`, but cap it at `_max_speed`. If capped, print `"Max speed reached! Speed capped at [max_speed]"`.
3. Implement `get_current_speed()` to return current speed.

SAMPLE INPUT

```
Model: "Sedan", Max Speed: 150 Actions: - accelerate(100) - accelerate(80) - get_current_speed()
```

SAMPLE OUTPUT

```
Max speed reached! Speed capped at 150 150
```

Task 7: Shopping Cart Wallet

Private Collections

PROBLEM STATEMENT

Encapsulate list modifications within an e-commerce shopping cart:

1. Create a class `ShoppingCart` with private list attribute `__items` (initially empty).
2. Implement `add_item(item_name, price)` storing item as a dictionary `{"name": item_name, "price": price}` inside `__items`.
3. Implement `remove_item(item_name)` to remove item if present, else print `"Item not found"`.
4. Implement `get_total_price()` returning sum of prices of all items in `__items`.

SAMPLE INPUT

```
Actions: - add_item("Book", 15.0) - add_item("Pen", 2.5) - remove_item("Eraser") - get_total_price()
```

SAMPLE OUTPUT

```
Item not found 17.5
```

Task 8: Credit Card Limit Protection

Property Decorator

PROBLEM STATEMENT

Manage credit limits and balance using encapsulated attributes:

1. Create a class `CreditCard` initializing with `card_number`, private `__credit_limit`, and private `__balance` (default 0.0).
2. Define property `balance` (getter only).
3. Implement `charge(amount)` adding to `__balance` if `__balance + amount <= __credit_limit`, otherwise print `"Transaction declined: Credit limit exceeded"`.
4. Implement `make_payment(amount)` reducing `__balance`.

SAMPLE INPUT

```
Card: "4111..", Limit: 1000.0 Actions: - charge(700.0) - charge(400.0) - make_payment(300.0) - charge(400.0)
```

SAMPLE OUTPUT

```
Transaction declined: Credit limit exceeded New balance: 800.0
```

Task 9: Smart Home Thermostat

Getters & Setters

PROBLEM STATEMENT

Encapsulate thermostat temperature settings with operating safety bounds:

1. Create a class `Thermostat` initializing with private `__target_temp` (default 20.0).
2. Implement `get_target_temp()` returning current target temperature.
3. Implement `set_target_temp(temp)` restricting range between 15.0°C and 30.0°C inclusive. If out of range, print `"Target temperature must be between 15°C and 30°C"`.

SAMPLE INPUT

```
Initial Target: 20.0 Actions: - set_target_temp(10.0) - set_target_temp(24.5) - get_target_temp()
```

SAMPLE OUTPUT

```
Target temperature must be between 15°C and 30°C 24.5
```

Task 10: Patient Medical Record

Data Hiding & Read-Only

PROBLEM STATEMENT

Design a confidential patient record class with selective read-only and write permissions:

1. Create a class `PatientRecord` initializing with private `__patient_id`, public `name`, and private list `__medical_history`.
2. Define a read-only property `patient_id` returning `__patient_id` (no setter allowed).
3. Implement `add_diagnosis(diagnosis)` appending a new record to `__medical_history`.
4. Implement `get_history(doctor_passcode)` returning `__medical_history` list if `doctor_passcode == "DOC123"`, else returning `"Access Denied"`.

SAMPLE INPUT

```
ID: "P992", Name: "Eva" Actions: - add_diagnosis("Flu") - get_history("WRONG") - get_history("DOC123")
```

SAMPLE OUTPUT

```
Access Denied ['Flu']
```

Encapsulation Rules & Naming Conventions

- **Private Attributes (Double Underscore):** Use `__attribute` prefix to trigger name mangling and block direct external access.
- **Protected Attributes (Single Underscore):** Use `_attribute` prefix to signify internal/subclass use by convention.
- **Property Decorators:** Prefer Pythonic `@property` and `@ .setter` for dynamic validation and controlled access over raw instance variables.
- **Validation Control:** Mutator methods (setters) must validate incoming arguments before altering internal class state.